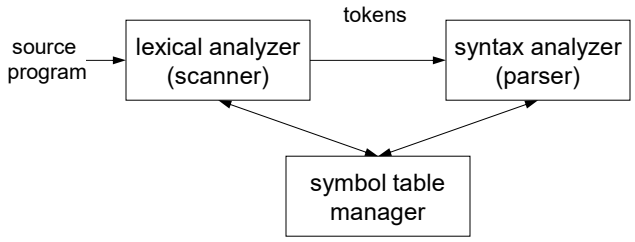
Lexical Analysis

1

1

Overview



- *Main task*: to read input characters and group them into "*tokens*."
- *Secondary tasks*:
  - Skip comments and whitespace;
  - Correlate error messages with source program (e.g., line number of error).

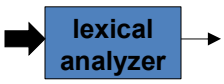
2

2

Overview (cont'd)

Input file

/	*	p	g	m	.	c	*	/	\n	i	n
t		m	a	i	n	(	i	n	t		a
r	g	c	,	c	h	a	r	*	*	a	r
g	v	)	{	\n	\t	i	n	t		x	,
Y	;	\n	\t	f	l	o	a	t		w	;
. . .											



Token sequence

keywd_int
identifier: "main"
left_paren
keywod_int
identifier: "argc"
comma
keywd_char
star
star
identifier: "argv"
right_paren
left_brace
keywd_int
…

3

3

The scanning process

- Main goal: recognize words
- How? by recognizing patterns
  - e.g. an identifier is a sequence of letters or digits that starts with a letter.
- Lexical patterns form a regular language
- Regular languages are described using regular expressions (REs)

4

4

Implementing Lexical Analyzers

Different approaches:

- Using a scanner generator, e.g., **lex** or **flex**. This automatically generates a lexical analyzer from a high-level description of the tokens.

  (easiest to implement; least efficient)
- Programming it in a language such as C, using the I/O facilities of the language.

  (intermediate in ease, efficiency)
- Writing it in assembly language and explicitly managing the input.

  (hardest to implement, but most efficient)

5

5

Lexical Analysis: Terminology

- ***token***: a name for a set of input strings with related structure.

  Example: "identifier," "integer constant"
- ***pattern***: a rule describing the set of strings associated with a token.

  Example: "a letter followed by zero or more letters, digits, or underscores."
- ***lexeme***: the actual input string that matches a pattern.

  Example: `count`

6

6

Examples

Input: `count = 123`

Tokens:

identifier : *Rule*: "letter followed by …"

Lexeme: count

assg_op : *Rule*: =

Lexeme: =

integer_const : *Rule*: "digit followed by …"

Lexeme: 123

7

7

Attributes for Tokens

- If more than one lexeme can match the pattern for a token, the scanner must indicate the actual lexeme that matched.
- This information is given using an *attribute* associated with the token.

Example: The program statement

`count = 123`

yields the following token-attribute pairs:

⟨*identifier*, pointer to the string **"count"**⟩

⟨*assg_op*, ⟩

⟨*integer_const*, the integer value **123**⟩

8

8

Specifying Tokens: regular expressions

- ***Terminology:***
  alphabet : a finite set of symbols
  string : a finite sequence of alphabet symbols
  language : a (finite or infinite) set of strings.
- ***Regular Operations on languages***:
  Union: $R \cup S = \{ x \mid x \in R$ or $x \in S\}$
  Concatenation: $RS = \{ xy \mid x \in R$ and $y \in S\}$
  Kleene closure: R* = R concatenated with itself 0 or more times
  $= \{\varepsilon\} \cup R \cup RR \cup RRR \cup$
  = strings obtained by concatenating a finite number of strings from the set R.

9

9

The scanning process

- Definition: Regular expressions (over alphabet Σ)
  - ε is an RE denoting $\{\varepsilon\}$
  - If $\alpha \in \Sigma$, then α is an RE denoting $\{\alpha\}$
  - If r and s are REs, then
    - (r) is an RE denoting L(r)
    - r|s is an RE denoting $L(r) \cup L(s)$
    - rs is an RE denoting L(r)L(s)
    - r* is an RE denoting the Kleene closure of L(r)

10

10

The scanning process

- Definition: Deterministic Finite Automaton
  - a five-tuple (Σ, S, δ, s_0, F) where
    - Σ is the alphabet
    - S is the set of states
    - δ is the transition function ($S \times \Sigma \rightarrow S$)
    - s_0 is the starting state
    - F is the set of final states ($F \subseteq S$)
- Notation:
  - Use a transition diagram to describe a DFA
- DFAs are equivalent to REs

11

11

The scanning process

- Goal: automate the process
- Idea:
  - Start with an RE
  - Build a DFA
    - How?
      - We can build a non-deterministic finite automaton (Thompson's construction)
      - Convert that to a deterministic one (Subset construction)
      - Minimize the DFA (Hopcroft's algorithm)
  - Implement it
- Existing scanner generator: flex

12

12

Characters vs Tokens (review)

- Input text

```
// this statement does very little
if (x >= y) y = 42;
```

- Token Stream

IF | LPAREN | ID(x) | GEQ | ID(y)

RPAREN | ID(y) | BECOMES | INT(42) | SCOLON

13

13

Why Separate the Scanner and Parser?

- Simplicity & Separation of Concerns
  - Scanner hides details from parser (comments, whitespace, input files, etc.)
  - Parser is easier to build; has simpler input stream (tokens)
- Efficiency
  - Scanner can use simpler, faster design
    - (But still often consumes a surprising amount of the compiler's total execution time)

14

14

Tokens

- Idea: we want a distinct token kind (lexical class) for each distinct terminal symbol in the programming language
  - Examine the grammar to find these
- Some tokens may have attributes
  - Examples: integer constant token will have the actual integer (17, 42, …) as an attribute; identifiers will have a string with the actual id

15

15

Typical Tokens in Programming Languages

- Operators & Punctuation
  - + - * / ( ) { } [ ] ; : :: < <= == = != ! …
  - Each of these is a distinct lexical class
- Keywords
  - if while for goto return switch void …
  - Each of these is also a distinct lexical class (*not* a string)
- Identifiers
  - A signle ID lexical class, but parameterized by actual id
- Integer constants
  - A single INT lexical class, but parameterized by int value
- Other constants, etc.

16

16

Principle of Longest Match

- In most languages, the scanner should pick the longest possible string to make up the next token if there is a choice
- Example

  return maybe != iffy;

  should be recognized as 5 tokens

RETURN	ID(maybe)	NEQ	ID(iffy)	SCOLON

i.e., != is one token, not two, "iffy" is an ID, not IF followed by ID(fy)

17

17

Formal Languages & Automata Theory (a review in one slide)

- Alphabet: a finite set of symbols
- String: a finite, possibly empty sequence of symbols from an alphabet
- Language: a set, often infinite, of strings
- Finite specifications of (possibly infinite) languages
  - Automaton – a recognizer; a machine that accepts all strings in a language (and rejects all other strings)
  - Grammar – a generator; a system for producing all strings in the language (and no other strings)
- A particular language may be specified by many different grammars and automata
- A grammar or automaton specifies only one language

18

18

Regular Expressions and FAs

- The lexical grammar (structure) of most programming languages can be specified with regular expressions
  - (Sometimes a little cheating is needed)
- Tokens can be recognized by a deterministic finite automaton
  - Can be either table-driven or built by hand based on lexical grammar

19

19

Regular Expressions

- Defined over some alphabet Σ
  - For programming languages, alphabet is usually ASCII or Unicode
- If *re* is a regular expression, *L*(*re*) is the language (set of strings) generated by *re*

20

20

Fundamental REs

re	*L*(*re*)	Notes
a	{ a }	Singleton set, for each a in Σ
ε	{ ε }	Empty string
∅	{ }	Empty language

B-21

21

Operations on REs

re	*L*(*re*)	Notes
rs	L(r)L(s)	Concatenation
r|s	L(r) ∪ L(s)	Combination (union)
r*	L(r)*	0 or more occurrences (Kleene closure)

- Precedence: * (highest), concatenation, | (lowest)
- Parentheses can be used to group REs as needed

22

22

Abbreviations

- The basic operations generate all possible regular expressions, but there are common abbreviations used for convenience. Typical examples:

Abbr.	Meaning	Notes
r+	(rr*)	1 or more occurrences
r?	(r | ε)	0 or 1 occurrence
[a-z]	(a|b|…|z)	1 character in given range
[abxyz]	(a|b|x|y|z)	1 of the given characters

23

23

Examples

re	Meaning
+	single + character
!	single ! character
=	single = character
!=	2 character sequence
<=	2 character sequence
xyzzy	5 character sequence

B-24

24

More Examples

re	Meaning
[abc]+	
[abc]*	
[0-9]+	
[1-9][0-9]*	
[a-zA-Z][a-zA-Z0-9_]*	

B-25

25

Example

- Possible syntax for numeric constants

digit ::= [0-9]

digits ::= *digit*+

number ::= *digits* (. *digits*)?

([eE] (+ | -)? digits) ?

26

26

Recognizing REs

- Finite automata can be used to recognize strings generated by regular expressions
- Can build by hand or automatically
  - Not totally straightforward, but can be done systematically
  - Tools like Lex, Flex, Jlex do this automatically, given a set of REs

27

27

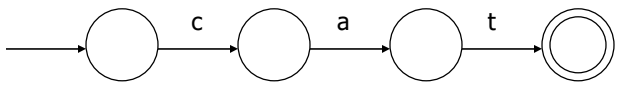
Finite State Automaton

- A finite set of states
  - One marked as initial state
  - One or more marked as final states
  - States sometimes labeled or numbered
- A set of transitions from state to state
  - Each labeled with symbol from Σ, or ε
- Operate by reading input symbols (usually characters)
  - Transition can be taken if labeled with current symbol
  - ε-transition can be taken at any time
- Accept when final state reached & no more input
  - Scanner uses a FSA as a subroutine – accept longest match each time called, even if more input; i.e., run the FSA from the current location in the input each time the scanner is called
- Reject if no transition possible, or no more input and not in final state (DFA)

28

28

Example: FSA for “cat”



29

29

DFA vs NFA

- Deterministic Finite Automata (DFA)
  - No choice of which transition to take under any condition
- Non-deterministic Finite Automata (NFA)
  - Choice of transition in at least one case
  - Accept if some way to reach final state on given input
  - Reject if no possible way to final state

30

30

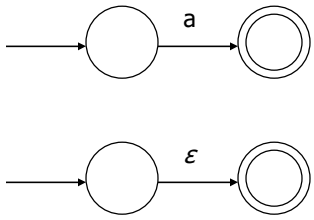
FAs in Scanners

- Want DFA for speed (no backtracking)
- Conversion from regular expressions to NFA is easy
- There is a well-defined procedure for converting a NFA to an equivalent DFA

31

31

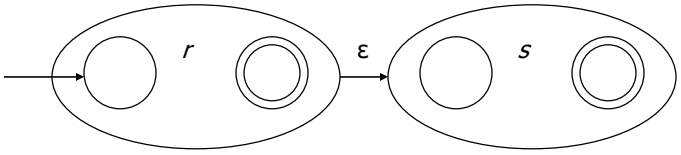
From RE to NFA: base cases



32

32

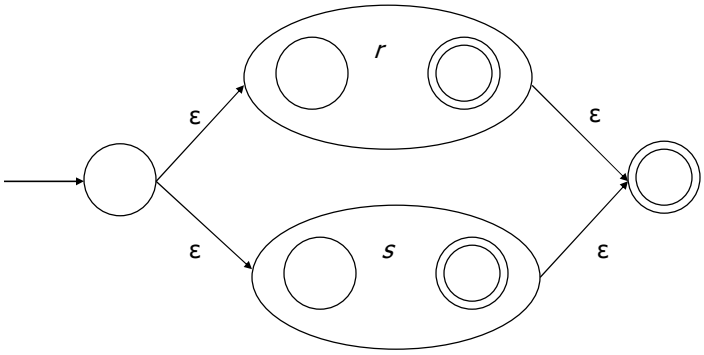
rs



33

33

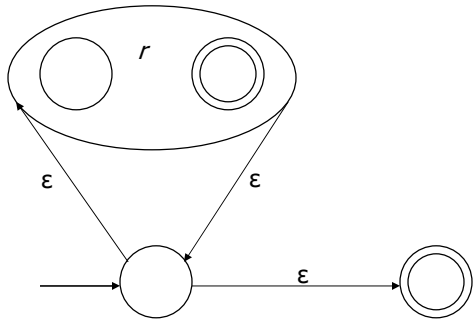
r | *s*



34

34

r *



35

35

From NFA to DFA

- Subset construction
  - Construct a DFA from the NFA, where each DFA state represents a *set* of NFA states
- Key idea
  - The state of the DFA after reading some input is the set of *all* states the NFA could have reached after reading the same input
- Algorithm: example of a fixed-point computation
- If NFA has *n* states, DFA has at most 2^n states
  - => DFA is finite, can construct in finite # steps
- Resulting DFA may have more states than needed
  - See books for construction and minimization details

36

36

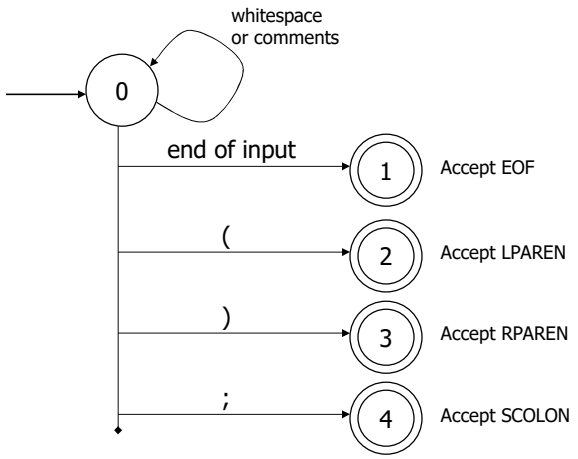
Example: DFA for hand-written scanner

- Idea: show a hand-written DFA for some typical programming language constructs
  - Then use to construct hand-written scanner
- Setting: Scanner is called whenever the parser needs a new token
  - Scanner stores current position in input
  - Starting there, use a DFA to recognize the longest possible input sequence that makes up a token and return that token

37

37

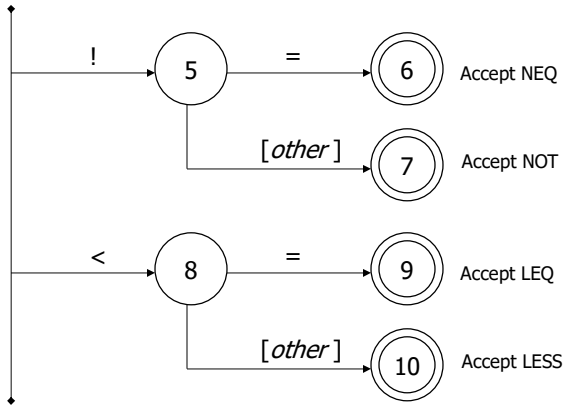
Scanner DFA Example (1)



38

38

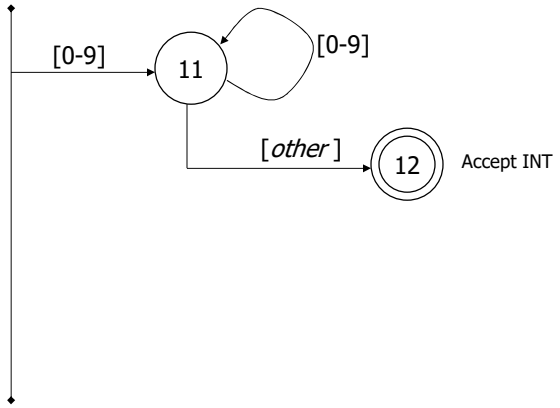
Scanner DFA Example (2)



39

39

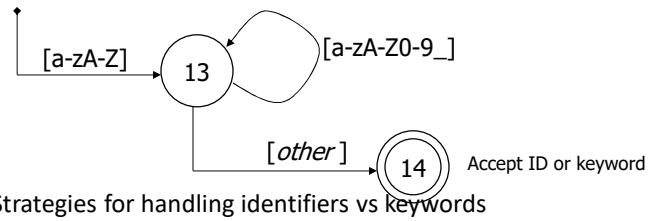
Scanner DFA Example (3)



40

40

Scanner DFA Example (4)



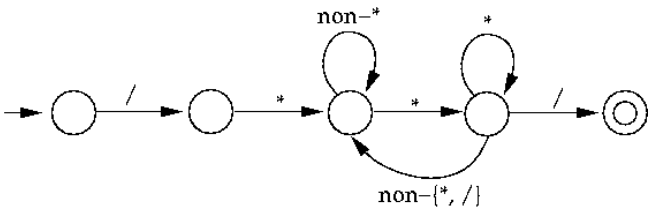
- Strategies for handling identifiers vs keywords
  - Hand-written scanner: look up identifier-like things in table of keywords to classify (good application of perfect hashing)
  - Machine-generated scanner: generate DFA will appropriate transitions to recognize keywords
    - Lots 'o states, but efficient (no extra lookup step)

41

41

Finite Automata: An Example

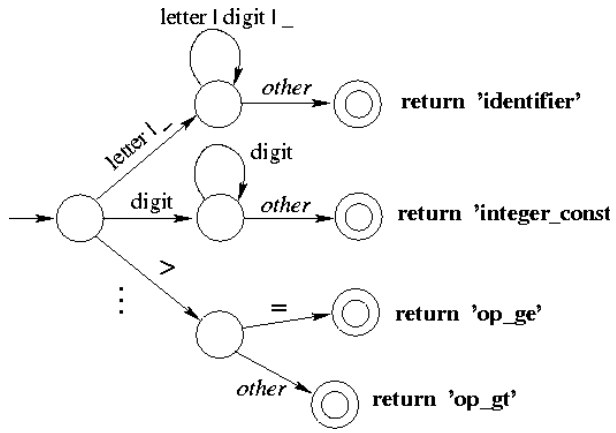
A (deterministic) finite automaton (DFA) to match C-style comments:



42

42

Structure of a Scanner Automaton



43

43

How much should we match?

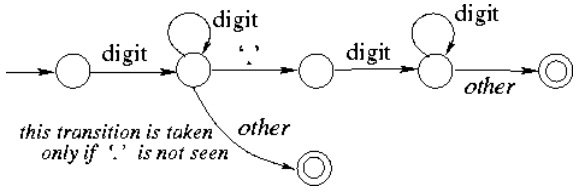
In general, find the longest match possible.

E.g., on input 123.45, match this as

num_const(123.45)

rather than

num_const(123), "**.**", num_const(45).



44

44

Implementing Finite Automata 1

Encoded as program code:

- each state corresponds to a (labeled code fragment)
- state transitions represented as control transfers.

E.g.:

```
while ( TRUE ) {
   …
   state_k: ch = NextChar();   /* buffer mgt happens here */
            switch (ch) {
              case … : goto  ...;     /* state transition */
                …
            }
   state_m:          /* final state */
            copy lexeme to where parser can get at it;
            return token_type;
   …
}
```

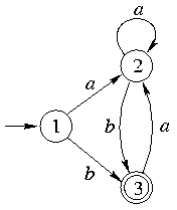
45

45

Table-Driven Automaton: Example

```
#define isFinal(s)      ((s) < 0)
int scanner()
{  char ch;
   int currState = 1;

   while (TRUE) {
      ch = NextChar( );
      if (ch == EOF) return 0;  /* fail */
      currState = T [currState, ch];
      if (IsFinal(currState)) {
         return 1;  /* success */
      }
   } /* while */
}
```



T

state \ input	*a*	*b*
1	2	3
2	2	3
3	2	-1

46

46

What do we do on finding a match?

- A match is found when:
  - The current automaton state is a final state; and
  - No transition is enabled on the next input character.
- Actions on finding a match:
  - if appropriate, copy lexeme (or other token attribute) to where the parser can access it;
  - save any necessary scanner state so that scanning can subsequently resume at the right place;
  - return a value indicating the token found.

47

47

Implementing a Scanner by Hand – Token Representation

- A token is a simple, tagged structure

```
public class Token {
   public int kind;              // token's lexical class
   public int intVal;  // integer value if class = INT
   public String id;             // actual identifier if class = ID
   // lexical classes
   public static final int EOF = 0;          // "end of file" token
   public static final int ID   = 1;         // identifier, not keyword
   public static final int INT = 2;          // integer
   public static final int LPAREN = 4;
   public static final int SCOLN   = 5;
   public static final int WHILE   = 6;
   // etc. etc. etc. …
```

48

48

Simple Scanner Example

```
// global state and methods

static char nextch;          // next unprocessed input character

// advance to next input char
void getch() { … }

// skip whitespace and comments
void skipWhitespace() { … }
```

49

49

Scanner getToken() method

```
// return next input token
public Token getToken() {
  Token result;

  skipWhiteSpace();

  if (no more input) {
      result = new Token(Token.EOF); return result;
  }

  switch(nextch) {
      case '(': result = new Token(Token.LPAREN); getch(); return result;
      case ')': result = new Token(Token.RPAREN); getch(); return result;
      case ';': result = new Token(Token.SCOLON); getch(); return result;

      // etc. …
```

50

50

getToken() (2)

```
case '!': // ! or !=
        getch();
        if (nextch == '=') {
          result = new Token(Token.NEQ); getch(); return result;
        } else {
          result = new Token(Token.NOT); return result;
        }

case '<': // < or <=
        getch();
        if (nextch == '=') {
          result = new Token(Token.LEQ); getch(); return result;
        } else {
          result = new Token(Token.LESS); return result;
        }
// etc. …
```

51

51

getToken() (3)

```
case '0': case '1': case '2': case '3': case '4':
case '5': case '6': case '7': case '8': case '9':
        // integer constant
        String num = nextch;
        getch();
        while (nextch is a digit) {
          num = num + nextch; getch();
        }
        result = new Token(Token.INT, Integer(num).intValue());
        return result;
…
```

52

52

getToken (4)

```
case 'a': … case 'z':
case 'A': … case 'Z':  // id or keyword
        string s = nextch; getch();
        while (nextch is a letter, digit, or underscore) {
          s = s + nextch; getch();
        }
        if (s is a keyword) {
          result = new Token(keywordTable.getKind(s));
        } else {
          result = new Token(Token.ID, s);
        }
        return result;
```

53

53

The scanning process

- A simple hand-written scanner would look a bit like this:

```
…
nextchar = getNextChar();
switch (nextchar) {
      case '(':          return LPAREN;   /* return LPAREN token */
      case 0:
      case 1:
      ...
      case 9:            nextchar = getNextChar();
                         while (nextchar is a digit)   {
                            concat the digits to build an integer
                            nextchar = getNextChar();
                         }
                         putBack(nextchar)
                         make a new INTEGER token with the integer  value attached
                         return INTEGER;
     ...
}
…
```

54

54